

Cloud Computing Lab 2: Integration with AWS

Due Date

Wednesday, September 17, 2025 11:59 PM

Objectives

In this lab, you will learn about AWS, how to set up an EC2 instance and S3 buckets, and the methods to integrating them into a FastAPI python project. Specifically, you'll be re-developing your Lab 1 API to work on an EC2 instance and taking data from S3 storage.

What is AWS?

AWS is a subsidiary of Amazon that provides cloud computing services and APIs, two of which we will use in this lab. AWS capitalizes on scalable and inexpensive services, boasting that you "only pay for what you need". Thankfully, there is a free tier that provides some services for a year, of which we will take full advantage. The services we'll use are:

- **EC2** - Short for **Elastic Cloud Compute**, EC2 is a virtual server service that AWS provides. Practically working as your own virtual computer, EC2 allows you to host websites, process large-scale data, and host databases, among many other things.
- **S3** - Amazon's **Simple Storage Service**, S3 is a secure place to store data that can be used from anywhere, given the target has permission of course.

Setting Up Your EC2 Instance

1. Setting up AWS
 - (a) Create AWS Account
 - (b) Sign up for EC2 Web Service
 - (c) Log in to AWS Management Console
2. Creating EC2 Instance
 - (a) Navigate to the EC2 Dashboard
 - (b) Find **Launch Instance** button and click it
 - (c) Under **Application and OS Images (Amazon Machine Image)**
 - i. Select *Amazon Linux*
 - ii. Select *Amazon Linux 2023 (AMI)*
 - iii. All other options can stay default
 - (d) Under **Instance type**
 - i. Select *t2.micro*

- (e) Under **Key pair (login)**
 - i. Create new key pair, selecting *RSA* as the key pair type and *.pem* as the key file format. Name it whatever you'd like and hit **Create key pair**
 - ii. NOTE: Make sure you know where this file is downloaded, you'll need it later
 - (f) Under **Network settings**
 - i. Make sure you're auto-assigning the public IP
 - ii. Select "Create security group", and select all types of connections (SSH, HTTPS, HTTP)
 - iii. Allow all IPs (Anywhere - 0.0.0.0/0) to access these connections
 - (g) Under **Configure storage**
 - i. All defaults are fine
3. You're now taken to the instances dashboard, displaying all currently running instances (which can be changed to all instances)
4. Configuring EC2 Instance
- (a) Click on the instance ID you just created and look at the instance details window at the bottom
 - (b) Go to the **Security** tab and click on the security group for this instance under *Security groups*
 - (c) You should now see the details of the security group. Scroll down to the **Inbound rules** table and click **Edit inbound rules**
 - (d) Click **Add rule** and give it the following settings
 - Type - Custom TCP
 - Port range - 8000
 - Source - Anywhere-IPv4
 - (e) These settings allow us to access our FastAPI on the uvicorn server we set up on our instance later
 - (f) Click **Save rules**
 - (g) Go back to the page showing all the details of your instance
 - (h) Click on the **Actions** dropdown near the top, go to the **Security** sub-menu, and click on **Modify IAM Role**
 - (i) Click the **Create new role** button, which will open a new tab. Keep the previous tab open for later
 - (j) You're now in the IAM Console. Click the **Create role** button
 - (k) Select **AWS service** as the *Trusted entity type* and **EC2** as the *Use case*. Click **next**.
 - (l) In the *Permissions policies* search box, find **AmazonS3ReadOnlyAccess** and select it. Click **next**.
 - (m) Name the role something relevant ("427-role" or something), description as whatever, and click the **Create role** button near the bottom, keeping all other settings default.
 - (n) Go back to the **Modify IAM Role** tab and refresh it. Now when you try to select a role, this newly-created role should show up. Select it and set it as the role for your instance. This allows files in your instance to access S3 without any sort of credential configuration.
NOTE: If you ever experience any sort of AWS configuration/credential error, reboot your instance (from the EC2 console) and wait for the instance to boot back up. If you have any uvicorn server or docker container (more on that later) running, you'll need to restart those.
5. Connecting to your EC2 Instance
- There are two ways to go about this. One easy way is to just use AWS' built-in terminal, which is a really handy feature they recently added, and the way that the writer of this document prefers due to ease.

Another way is to directly ssh into your instance through your own terminal.
If you're on an Apple Machine, you can just open up the terminal app in order to do so.
If you're on a Windows Machine, you'll need to download a Linux terminal like Cygwin

6. SSH into your EC2 Instance (if not using AWS' built-in terminal)

- (a) Open your Terminal app
- (b) Navigate to the directory in which the key pair `.pem` file is stored.
- (c) Use the command:

```
1 > ssh -i <privatekey.pem> ec2-user@ec2-<numbers>.us-east-2.compute.amazonaws.com
```

7. Regardless of how you connect to your instance, you should now see some ASCII art telling you you're in the EC2 instance! We've gotta install some stuff now.

- (a) To install the python package manager, run

```
1 > sudo yum install python3-pip
```

- (b) Install `venv` package through `pip3`, which will allow us to create a virtual environment for our Python code

```
1 > pip3 install virtualenv
```

- (c) We also need to install `tmux`, which is a Terminal Multiplier, i.e. how we can have a terminal running in the background (more on that later)

```
1 > sudo yum install tmux
```

Setting Up S3

1. Navigate to the S3 console
- 2.
3. Click the "Create bucket" button, name it something relevant (like "cse-427"), and keep all the default options
4. Now that the bucket is created, navigate into it and create a new folder. I named mine "lab2"
5. We're going to add some files to this folder. For testing, we'll use the `users.json` document we gave you in the last lab. Eventually, you'll be using the `movies.json` that was also present in the last lab.

No other configuration should be needed outside of this. Because we set our EC2 instance to have the IAM user that can access all S3 services, when we run programs directly in the instance, you should have no problem with authentication. If you have any problems with it, feel free to ask your TAs or Instructors.

Finally, Some Coding

We need to do a little more setup to get our API working, which will require the virtual environment package we installed earlier. So, navigate to your working directory and follow these steps:

1. Create a file named `requirements.txt` that will house the modules you need for your api:

```
1 > touch requirements.txt
```

2. Edit this file with your favorite text editor (I'll be using vim):

```
1 > vi requirements.txt
```

3. You'll most likely be using this text editor a lot, so be sure to get familiar with it and its commands. Vim, for example, has a lot of commands and shortcuts, so check out a tutorial like <https://www.openvim.com>. When you're somewhat comfortable with your editing environment, copy the following lines into your file, which represent the modules needed:

```
1 fastapi
2 uvicorn
3 jinja2
4 boto3
```

You'll notice that three we've used before: `fastapi`, `uvicorn`, and `jinja2`. However, the fourth, `boto3` gives us methods to access the data in our S3 bucket.

Our next step would be to install these packages. However, it'd be good to set up a virtual environment first:

1. You could install these packages straight-up, or you could create a virtual environment to house them instead, which is what we'll do. To create this environment, use the command:

```
1 > python3 -m venv env
```

This creates a virtual environment directory called `env` in your current folder.

2. Activate the virtual environment:

```
1 > source env/bin/activate
```

You should now see a little `(env)` next to the IP/user of your command line, which means you've activated your virtual environment!

3. Install the packages in `requirements.txt`:

```
1 > pip3 install -r requirements.txt
```

Now, to demonstrate how `boto3` can communicate with S3, we're going to set up a new file. This file won't be an exact copy for what you'll do for the DIY part of this lab, as it will use the `users.json` data, and not actually use an API. However, it will show you how to get data from S3 and convert it to JSON so that you can use it in the same way as lab 1, just on your EC2 instance!

1. Create a new python file called `data.py`
2. Edit this file to include the following code. I would recommend not just copying and pasting it, as it helps to type code out so you can have some repetitions of methods/common practices in your fingers.

```
1 import boto3
2 import json
3
4 s3 = boto3.resource("s3") # the object that will help you read data from S3
5
6 # Gets data from the file "users.json"
7 # which is within the folder "lab2"
8 # which is within the bucket "cse-427"
9 def get_data():
10     obj = s3.Object("cse-427", "lab2/users.json") # gets the file contents
```

```
11     body = obj.get()['Body'].read() # reads the file contents as a string
12     json_data = json.loads(body) # converts string into json
13     return json_data # returns json from file
14
15 print(get_data()) # calls and prints returned json data
```

3. Running this file with the command `> python data.py` will print out the data that was in the json file.
This method can be similarly used in your `main.py` in order to get the data given to you for the lab. You can get this data from S3, then return it from one of your own API endpoints in an HTML template.

Running Your Server

Obviously, the python file we just created isn't the same as your API. Your API was using `uvicorn` to run a server, which put your API on localhost at port 8000. However, if you run your server as is, you might notice two things:

1. If you close your terminal, that server will stop, shutting down your API and making us unable to grade your work.
2. You can't find your API on the IP address that it gives you, as it's still giving you your localhost (`http://127.0.0.1:8000`).

So, we need to specify how `uvicorn` is going to set up its server by adding tags to the command:

```
1 > uvicorn --host 0.0.0.0 --port 8000 main:app
```

This tells the server that it will be run on any IP address (i.e. your instance's public IP address) on port 8000. Running this and navigating to your instance's public IP address at the port 8000 will allow you to view your endpoints like normal. Now, the above solves the second problem, but what about the first? That's where `tmux` comes in. Running the command:

```
1 > tmux
```

will clear your console and set up a new terminal for you to work in, completely separate but running concurrently with the one you just left. So, you can run your server over here with the same `uvicorn` command, new tags and all. Then, navigate back to your original terminal by hitting Ctrl+B then D. Now, when you run commands or exit your ssh, your server will still be running.

DIY

All that you're going to do is implement the same API to show movies, except now all your data should be located in S3 and we can access your API from your instance's public IP address.

Additional Software Installation

This lab also requires installing software that we'll use in later labs: Docker Desktop and minikube. After installing both, run the following commands from the command line:

```
> docker --version
```

```
> minikube start; minikube status
```

The output of the docker command should display the version installed
The output of the minikube status command should display the following:

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Paste both outputs into a text file named softwareInstallation.txt and push the file to your lab 2 GitHub repo.

Grading

- 5 pts - Setup AWS Instance
- 5 pts - Create IAM Role
- 5 pts - Create S3 Bucket
- 20 pts - Deploy Movies API on AWS
- 2 pts - Installation of Docker Desktop
- 3 pts - Installation of minikube

Submission

Submit this assignment to your lab 2 repo on GitHub before the due date. Please include a link to your ec2 instance in the README.md.